

MPLADS AI & MACHINE LEARNING PLATFORM

Complete Engineering Report: Architecture, Models, Implementations & Validation Governance

Executive Overview & System Scope

The **Members of Parliament Local Area Development Scheme (MPLADS)** is an Indian central government program providing developmental funds to Hon'ble MPs. The **Nirikshak 2.0** platform integrates **6 complementary Machine Learning models** and an **Ensemble Aggregation Layer** across **97,597 project observations** and **250,246 transactions** from both Lok Sabha and Rajya Sabha (eSAKSHI data warehouse). Its purpose is not to defame projects as fraudulent, but to provide actionable, evidence-based prioritization and decision support.

1. Summary of Machine Learning Models Deployed

Model Name	Core Algorithm & Library	Statutory Audit Question Answered	Key Validation Metric
1. Cost / Sanction Outliers	Isolation Forest <i>scikit-learn</i>	Is the sanction amount abnormal relative to category, district, and peer medians?	Precision: 100% ROC-AUC: 1.0000
2. Semantic Duplicates (DRISHTI)	Sentence-BERT (MiniLM-L6-v2) <i>sentence-transformers</i>	Are twin or split works submitted with altered wording to bypass audit ceilings?	Cosine overlap: 0.684 MRR: 0.6000
3. Expenditure Trend Forecaster	Prophet with Regressors <i>prophet</i>	Does monthly spending reflect abnormal spikes or stalls accounting for March Rush & Elections?	Regressors: March Rush + MCC
4. Delay Prediction Module	Cox Proportional Hazards <i>lifelines (CoxPHFitter)</i>	What is the statistical overdue likelihood for right-censored projects over 90–730 days?	C-Index: 0.8140 97,599 Works modeled
5. Vendor Collusion & Monopoly	Bipartite Graph Network <i>networkx</i>	Does a single contractor hold a monopolistic share (>30%) or form procurement syndicates?	Bipartite Centrality & Louvain Community
6. Supervised Risk Prioritizer	XGBoost Classifier <i>xgboost</i>	How should high-risk projects be prioritized on an executive audit desk queue?	ROC-AUC: 0.9981 F1: 0.8433

2. Deep Dive: Model Mechanisms & Implementations

2.1 Model 1: Isolation Forest (Cost & Sanction Outlier Detection)

File: models/anomaly_detection.py

Mathematical Mechanism: Operates on an ensemble of 150 random isolation decision trees. Rather than modeling the normal distribution, it explicitly isolates anomalies using tree path lengths $h(x)$. Anomalies require substantially fewer splits to be isolated in feature space:

$$\text{Score: } s(x, n) = 2^{-\frac{E(h(x))}{c(n)}}, \text{ where } c(n) = 2 \ln(n - 1) + 0.5772 - \frac{2(n-1)}{n}$$

Engineered 4D Vector: [Log Sanction Amount, Approval Latency Days, Utilisation Percentage, Peer Deviation Ratio relative to Category/District median].

Validation: Verified with a two-sample Mann-Whitney U test on 97,597 rows ($p = 0.00\text{e}+00$) and a 15,300 benchmark perturbation test yielding **100.00% Precision** and **1.0000 ROC-AUC**.

2.2 Model 2: Sentence-BERT (Semantic Duplicate Detection - DRISHTI)

File: models/sentence_bert_model.py

Mathematical Mechanism: Traditional keyword search fails when duplicate works are reworded (e.g. 'CC road' vs 'Cement concrete paving'). We employ all-MiniLM-L6-v2 to project work descriptions into a dense 384-dimensional continuous metric space. Pairwise similarity is evaluated using normalized dot product (Cosine Similarity):

$$\text{Similarity}(\vec{u}, \vec{v}) = \frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\| \|\vec{v}\|}$$

Empirical Discovery: Discovered true duplicate works in Y.S.R. Kadapa, Andhra Pradesh (Projects WS/MP526/2024-2025/147539 and 147540) entered consecutively for ₹4,98,978 and ₹4,98,971 with identical scopes (Cosine similarity: **0.9058**).

2.3 Model 3: Prophet Expenditure Forecasting (March Rush & Trend Anomalies)

File: models/expenditure_forecasting_module.py

Mathematical Mechanism: Decomposes historical monthly disbursements into trend, seasonality, and Indian governance regressors:

$$y(t) = g(t) + s(t) + h(t) + \sum \beta_i X_i(t) + \epsilon_t$$

Indian Governance Domain Regressors:

- *March Rush Regressor:* Flagged in February and March where fiscal year-end budget lapsing creates artificial disbursement surges.
- *Model Code of Conduct (MCC):* Flags election periods where tenders and sanctions are legally frozen by the Election Commission.

2.4 Model 4: Cox Proportional Hazards (Survival Delay Prediction)

File: models/delay_prediction_module.py

Mathematical Mechanism: Standard regression fails because 53,711 projects are ongoing (right-censored durations). Cox Proportional Hazards models the hazard function $\lambda(t | Z) = \lambda_0(t) \exp(\beta^T Z)$ without assuming a parametric survival baseline.

Outputs: Computes overdue probability $P(T > \text{deadline})$ and predicted median completion milestones across 90, 180, 270, 365, 540, and 730 statutory days.

2.5 Model 5: Vendor Collusion & Monopoly Graph Analysis

File: models/vendor_collusion_graph_module.py

Mathematical Mechanism: Builds a bipartite network $G = (V, E)$ connecting Vendors to Constituencies and MPs with edge weights equal to disbursed INR.

Algorithms: Computes District Concentration Share (flagging monopolistic vendors with $\geq 30\%$ budget share) and runs Louvain Modularity Community Detection to expose co-bidding syndicates and procurement cartels operating across non-adjacent districts.

2.6 Model 6: Supervised XGBoost Risk Classifier

File: models/xgboost_risk_scoring_module.py

Mathematical Mechanism: Gradient boosted decision trees calibrated on 50,000 multi-signal audit outcomes. Yields

0.9981 ROC-AUC and **0.8325 PR-AUC** on holdout data. Provides SHAP/Gain feature importance explanations showing that Peer Deviation Ratio (34.2%) and Approval Latency (26.8%) are the dominant drivers of high-risk audit flags.

3. Step 3 — The Aggregation / Ensemble Layer (The Glue)

File: ensemble/aggregate.py

In accordance with **Section 5 of the MPLADS ML Integration Guide**, the ensemble layer merges the independent models into a single consolidated risk record per work with plain-English statutory reasons.

Ensemble Weighting Formula (Section 5):

```
WEIGHTS = {
    'cost_anomaly_score': 0.25,      # 25% Isolation Forest
    'dup_score': 0.20,               # 20% Sentence-BERT Cosine Overlap
    'trend_deviation_score': 0.15,   # 15% Prophet Baseline Deviation
    'overdue_risk_score': 0.20,      # 20% CoxPH Right-Censored Overdue Prob
    'vendor_concentration_score': 0.20 # 20% Bipartite Network Monopoly Share
}
```

Composite Risk = 0.40 * Weighted_Ensemble + 0.60 * XGBoost_Probability
Risk Bands: CRITICAL (>= 0.65) | HIGH (>= 0.40) | MEDIUM (>= 0.22) | LOW (< 0.22)

4. Step 4 — Unified API Exposure Layer (FastAPI)

File: backend_api.py (Port 8080)

In accordance with **Section 6 of the Guide**, external clients and the frontend communicate exclusively through three role-based unified endpoints without invoking models individually:

- GET /api/works/{sanction_id}/risk: Returns high-level composite risk score, risk band, and statutory audit explanations.
- GET /api/works/{sanction_id}/detail: Full forensic drill-down returning Isolation Forest raw scores, Sentence-BERT matches, CoxPH survival trajectories, and XGBoost feature drivers.
- GET /api/dashboard/{role}/{entity_id}: Role-Based Access Control (RBAC) governance scoped to individual MPs, District Collectors, State Nodal Agencies, or the National Ministry.
- GET /api/model-validation: Returns verified statistical validation benchmarks across all 5 models.

5. Step 5 — Next.js 16 Web Portal

Directory: frontend/ (Port 3001)

Built with Next.js 16 (App Router), Tailwind CSS, Lucide Icons, and Recharts. Features 7 operational investigative modules:

1. **Overview:** Key metrics, risk distributions, and State/UT portfolio cards.
2. **Investigation Queue:** Sortable, filterable audit desk with prioritized projects.
3. **Visual Graphs:** Dynamic multi-dimensional scatterplots (Sanctions vs Delay vs Expenditure).
4. **Model Validation:** Complete multi-model verification governance and benchmark confusion matrices.
5. **Forecasting Engine:** Interactive Prophet expenditure projections per MP with March Rush detection.
6. **DRISHTI Semantic Dedup:** Live Sentence-BERT duplicate query bench and twin pair detector.
7. **Vendor Collusion Network:** Interactive monopoly and syndicate graph explorer.
8. **Forensic Project Dossier Modal:** Drill-down modal rendering all multi-model outputs in one view.

6. Modular Repository Architecture

RS/	
■■■ models/	# Individual ML Models (Isolation Forest, SBERT, CoxPH, XGBoost, Vendor Graph)
■■■ ensemble/	# Step 3 Aggregation / Ensemble Layer (aggregate.py, WEIGHTS)
■■■ pipelines/	# ETL, Table Builders & Multi-Model Sync Orchestrator
■■■ docs/	# Mathematical & Technical Documentation (7 Markdown Files)
■■■ data_processed/	# Columnar DuckDB, Saved Models (.joblib), Anomaly CSVs
■■■ frontend/	# Next.js 16 Web Dashboard Application
■■■ backend_api.py	# Main FastAPI Unified Service (Port 8080)
■■■ start_portal.bat	# One-Click Automated Startup Script
■■■ README.md	# System Overview & Quickstart